GUIA VISUAL 102

OBSERVABILIDADE E DESEMPENHO: STRACE, LTRACE E PERF

O serviço sobe, escreve `erro ao inicializar` e morre. O journal não tem mais nada, o autor sumiu em 2019 e não há fonte. ***Existe um recurso que funciona aqui e em quase todos os parentes deste caso:*** *em vez de perguntar ao programa o que houve, perguntar ao* ***kernel*** *o que o programa pediu.*

102
GUIAS LINUX

DO SINTOMA À FERRAMENTA — O MÉTODO VEM ANTES DO COMANDO

QUAL É O SINTOMA?

NÃO FUNCIONA
sem saída útil · falha, erro obscuro

- `strace -f -e trace=file` — arquivo que não abre, permissão negada
- `ltrace · ldd` — biblioteca errada, função que retorna erro

ESTÁ LENTO
saída correta · funciona, mas demora

- `perf top · perf record -g` — CPU alta → gráfico de chama
- `iostat · iotop · strace -c` — esperando E/S
- `/proc/PID/wchan · stack` — parado sem motivo

A classificação inicial custa trinta segundos e vem do `vmstat 1`: coluna `r` alta = disputa por CPU; colunas `b` e `wa` altas = espera por E/S; processo que não aparece em nenhuma das duas está **bloqueado** esperando travamento, rede ou outro processo. **O erro mais comum em diagnóstico de desempenho é escolher a ferramenta antes de saber que pergunta se está fazendo** — e a armadilha clássica que ele produz é otimizar o trecho errado. O `/proc/<PID>/wchan` nomeia a função do kernel em que o processo dorme e encerra a discussão em uma leitura: `futex_wait` = travamento entre threads, `sk_wait_data` = rede, `io_schedule` = disco.

CINCO CAMADAS, UMA PERGUNTA CADA — E AS OPÇÕES QUE RESOLVEM A MAIORIA

FUNÇÕES INTERNAS DO PROGRAMA só com símbolos ou fonte	`perf record -g` `gdb`
CHAMADAS A BIBLIOTECAS malloc, strcmp, SSL_read	`ltrace` `LD_PRELOAD`
CHAMADAS DE SISTEMA openat, read, connect, execve	`strace` `bpftrace`
EVENTOS DO KERNEL escalonamento, E/S de bloco, rede	`perf trace` `bpftrace · bcc`
CONTADORES DO PROCESSADOR ciclos, instruções, faltas de cache	`perf stat`

Escolher a camada errada custa horas. Programa travado esperando resposta de rede *não aparece* em `perf stat` — ele não gasta ciclo nenhum. Laço que consome CPU *não aparece* em `strace` — ele não faz chamada nenhuma. **Comece pela camada mais barata que possa conter a resposta e desça só se precisar.**

OPÇÃO DO STRACE	EFEITO
`-f`	segue processos filhos — quase sempre necessária
`-e trace=file`	só o que toca caminhos: `open` , `stat` , `access` . Também `network` e `%process`
`-c`	não rastreia: **resume** contagem, tempo e erros por chamada
`-p <PID>`	conecta a um processo já em execução
`-y · -T · -tt · -s 200`	o caminho de cada descritor · tempo dentro de cada chamada · horário com microssegundos · cadeias maiores (padrão: 32)

O `-c` vale como diagnóstico de desempenho: um programa que faz meio milhão de `read` de 1 byte não tem problema de disco — tem problema de tamanho de buffer, e a tabela do `-c` mostra isso em uma tela. **E o `ltrace` só enxerga o que atravessa a PLT:** binário estático não rende nada, e função interna do próprio programa também não aparece.

NA PRÁTICA — O CASO DA ABERTURA, E OBSERVAR SEM PARAR O PROGRAMA

kali@kali: ~

```
# a linha que resolve boa parte dos "nao funciona
# e nao diz por que":
$ strace -f -e trace=file -o /tmp/r.txt ./servico
$ grep -E 'ENOENT|EACCES|EPERM' /tmp/r.txt | tail
openat(AT_FDCWD, "/etc/servico/cfg.yml", O_RDONLY)
        = -1 ENOENT (No such file or directory)
# ENOENT=nao existe  EACCES=permissao  EPERM=proibido
$ strace -c ls -R /usr      # perfil agregado
$ strace -f -e trace=network -p $(pgrep -f servico)
# eBPF: roda dentro do kernel, entrega so o resultado
$ sudo execsnoop-bpfcc      # cada exec do sistema
$ sudo opensnoop-bpfcc      # cada open
$ sudo tcpconnect-bpfcc     # cada conexao de saida
# CPU: amostra 99x/s com pilha, e o grafico de chama
$ sudo perf record -F 99 -g -p <PID> -- sleep 30
```

Um roteiro que evita perder tempo: 1) reproduza — problema que não se reproduz não se mede; **2) classifique** com `vmstat 1` e `pidstat 1`: CPU, E/S ou bloqueio; **3) delimite** com `strace -c` ou `perf stat` — um número agregado antes de qualquer rastreio detalhado; **4) aprofunde na camada certa**; **5) mude uma coisa só** e meça de novo; **6) registre** — sem linha de base anotada, a próxima investigação recomeça do zero.

O `perf stat` responde ao que nenhuma outra responde: IPC baixo = CPU *esperando memória*, não calculando; `cache-misses` alto = estrutura com localidade ruim; `context-switches` alto = disputa ou E/S em excesso.

`strace -p` **respondeu** `Operation not permitted` **num processo do *seu próprio* usuário? A causa é o Yama, não permissão de arquivo.** `strace`, `ltrace` e `gdb` usam a mesma syscall `ptrace`, governada por `kernel.yama.ptrace_scope`: `0` = mesmo usuário, `1` = só pai e filho (padrão no Kali), `2` = só root, `3` = ninguém até reiniciar.

O `strace` desacelera o processo rastreado em uma ordem de grandeza — cada chamada interceptada custa duas trocas de contexto a mais. Em produção isso transforma um serviço lento num serviço **inoperante**, e há casos em que o atraso **altera o comportamento**: condições de corrida somem, tempos-limite estouram, e o problema que você foi investigar desaparece enquanto você observa. Prefira `-e trace=` restrito, use `-c` quando só quiser o agregado, e para observação contínua em produção use **eBPF**, que agrega dentro do kernel e entrega apenas o resultado.

RESUMO RÁPIDO

- ✓ **Duas perguntas, dois conjuntos.** "Por que não funciona?" → rastreamento (`strace` , `ltrace`). "Por que está lento?" → perfilamento (`perf` , `vmstat` , `pidstat`).
- ✓ **Todo programa passa pela fronteira da chamada de sistema** — e ali se observa *sem fonte, sem símbolos e sem cooperação* do software.
- ✓ `strace -f -e trace=file` + `grep ENOENT|EACCES|EPERM` resolve uma fração enorme dos "não funciona e não diz por quê": mostra o caminho exato que o programa procurou.
- ✓ **Cinco camadas, uma pergunta cada** — funções internas, bibliotecas, syscalls, eventos do kernel, contadores da CPU. Comece pela mais barata que possa conter a resposta.
- ✓ `/proc` **responde sem instalar nada:** `wchan` diz onde o processo dorme, e as trocas de contexto voluntárias × involuntárias separam "dorme esperando" de "disputa CPU".
- ✓ **eBPF é a fronteira atual:** roda um programa verificado *dentro* do kernel e entrega só o agregado — barato o bastante para produção.

NO KALI

O `strace` é a forma mais rápida de entender uma ferramenta que você não escreveu. Num script de exploração pouco documentado, `strace -f -e trace=file,network` mostra que arquivos ele lê, onde grava e com quem conversa na rede. Resolve dois problemas: **achar o relatório que a ferramenta gerou e verificar que uma ferramenta de repositório desconhecido não manda os seus dados para terceiros** — checagem obrigatória antes de rodar código alheio em ambiente de cliente. **Análise dinâmica de binário suspeito usa o mesmo conjunto, com regras extras:** amostra desconhecida só em VM isolada e descartável, snapshot antes e reversão depois — *nunca* em contêiner. Ali, `strace -f` , `ltrace` e o `opensnoop` / `tcpconnect` do eBPF revelam persistência, comunicação e leitura de credenciais. **Amostras modernas detectam** `ptrace` **e mudam de comportamento** — ver `ptrace(PTRACE_TRACEME)` no rastreio é o sinal de anti-depuração. **O** `ptrace_scope` **é botão dos dois lados:** `=1` (padrão do Kali) impede que um processo comprometido leia a memória de outro do mesmo usuário — inclusive a do navegador com as suas sessões abertas. `perf` **e eBPF exigem privilégio por bom motivo:** valor permissivo de `perf_event_paranoid` expõe temporização que serve a canal lateral — aceitável em laboratório, decisão de segurança num servidor compartilhado. **E rastrear não substitui análise estática — a ordem importa.** O roteiro seguro *começa sem executar nada*: `file` , `strings` , `readelf -d` , `objdump -d` e a soma de verificação contra fontes públicas. Só depois, e só isolado, vem a execução observada — inverter a ordem é executar primeiro e entender depois, que é o que o malware espera. Tudo em sistema próprio ou autorizado: invadir é crime (Lei 12.737/2012). **Na RHEL:** o pacote é `perf` , o `bcc-tools` instala em `/usr/share/bcc/tools/` sem o sufixo `-bpfcc` , e o `sosreport` coleta o diagnóstico inteiro num comando — vale imitar em qualquer distribuição antes de mexer num sistema com problema.